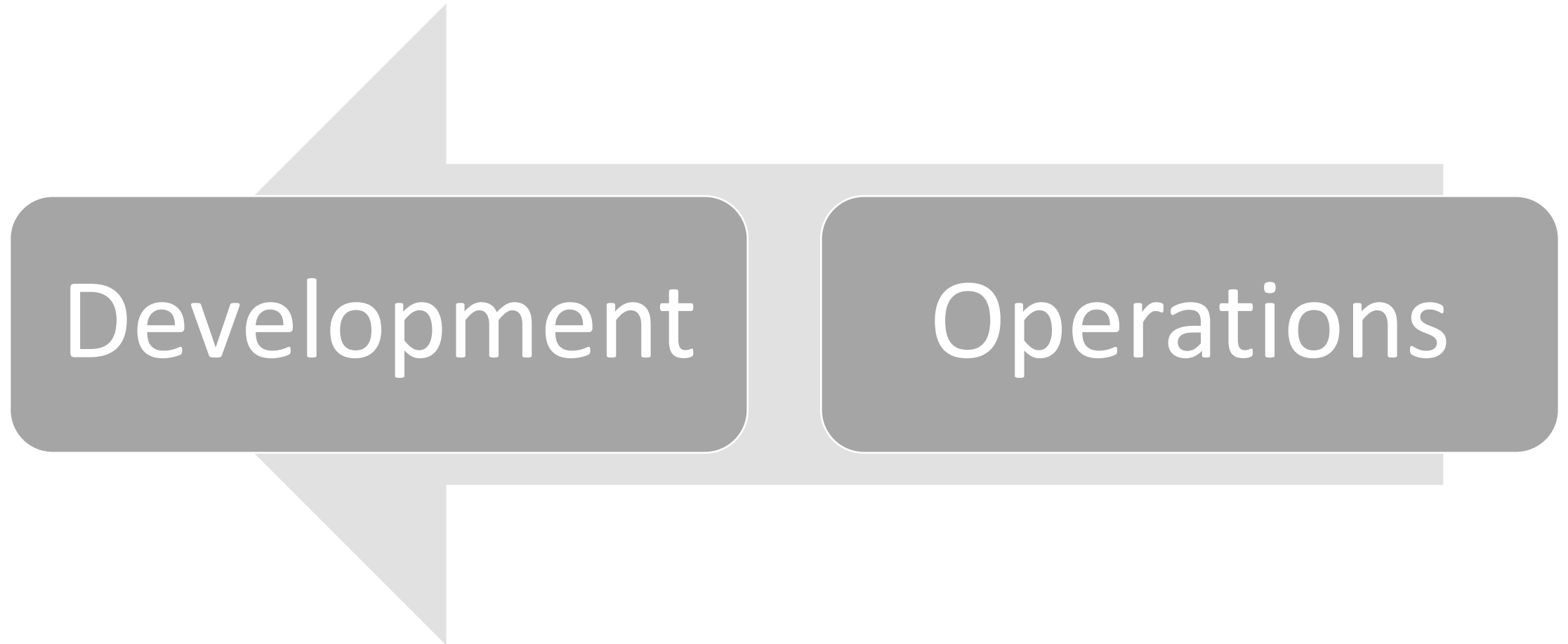




DevOps

2nd Way: Technical Practices of Feedback

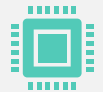
Continuous Fast Feedback Flow



Goals



Quickly find and fix problems earlier in the software development life cycle, ideally long before they cause a catastrophic failure



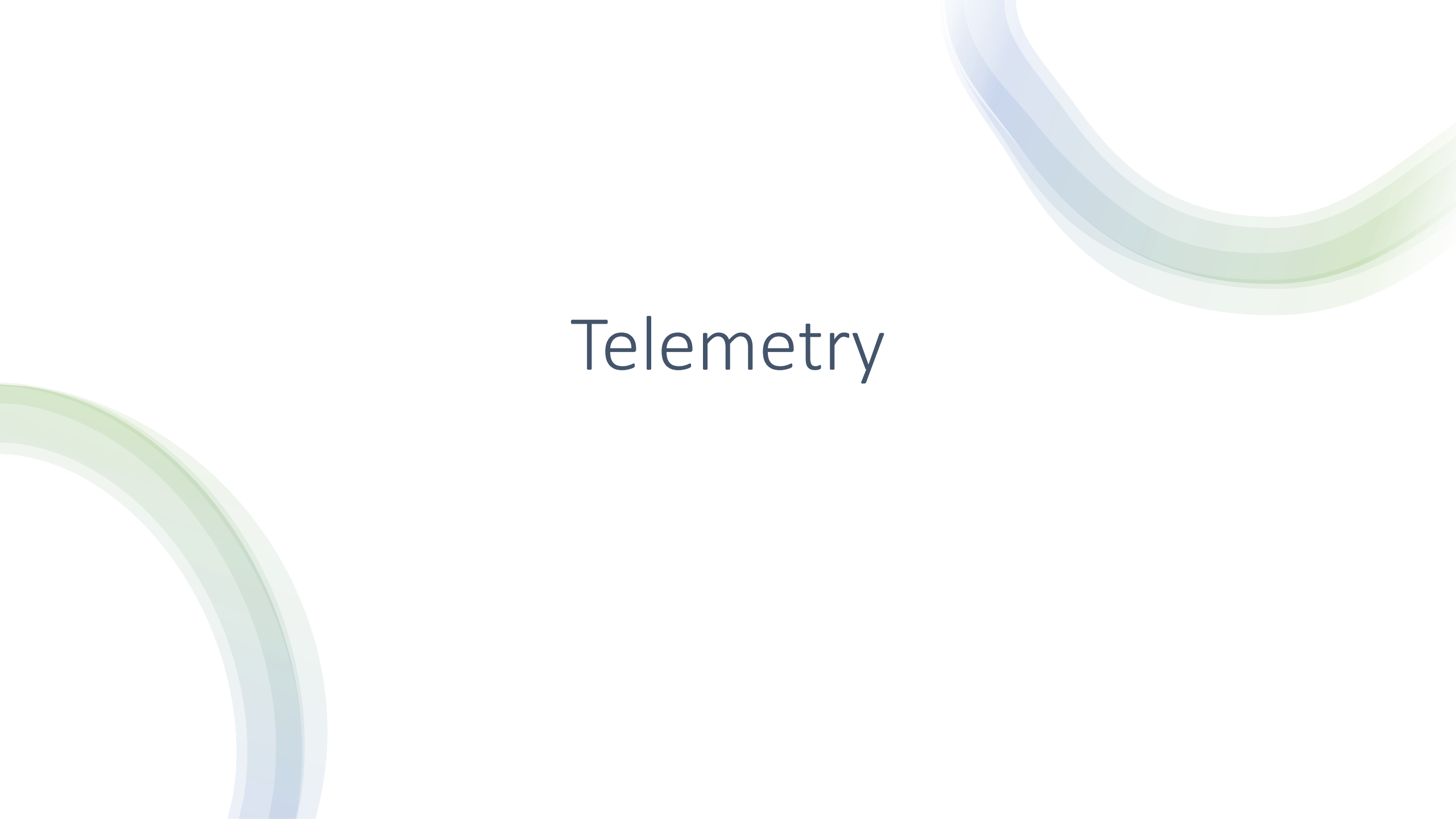
Create improvements and learnings, from production issue, deployment issue, early indicators of problems, or customer usage patterns



Rapidly test product hypotheses, to determine if the features we are building are helping us achieve our organizational goals



Create telemetry from build, test, and deploy processes, as well as from user behavior, production issues and outages, audit issues, and security breaches

The background features decorative curved lines in the corners. In the top-left and bottom-left corners, there are light green and blue curved lines. In the top-right corner, there is a thick, multi-layered curved line in shades of blue and green. The word "Telemetry" is centered in the middle of the page.

Telemetry

Create Telemetry to Enable Seeing and Solving Problems



A fact of life in Operations is that things go wrong—small changes may result in many unexpected outcomes, including outages and global failures that impact all our customers



Types of failures that can cause outages

Application

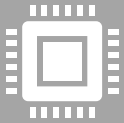
Environment

Something external (e.g., denial of service attack)

Disciplined problem-solving behavior



Design systems so that they are continually creating telemetry



Automated communications process by which measurements and other data are collected at remote points and are subsequently transmitted to receiving equipment for monitoring.



Create telemetry within applications and environments, both in production and pre-production environments as well as in deployment pipeline

Create Centralized Telemetry Infrastructure



Data collection at the business logic, application, and environments layer



Logs sent to a common service that enables easy centralization, rotation, and deletion



Gather metrics at all layers of the application stack to better understand how system is behaving (**Open Telemetry**)



An event router responsible for storing all events and metrics (DataDog, Azure Monitor, NewRelic, Prometheus)

Transform Centralized Logs into Metrics



Perform statistical
operations



Anomaly detection

Collect Telemetry from Deployment Pipeline

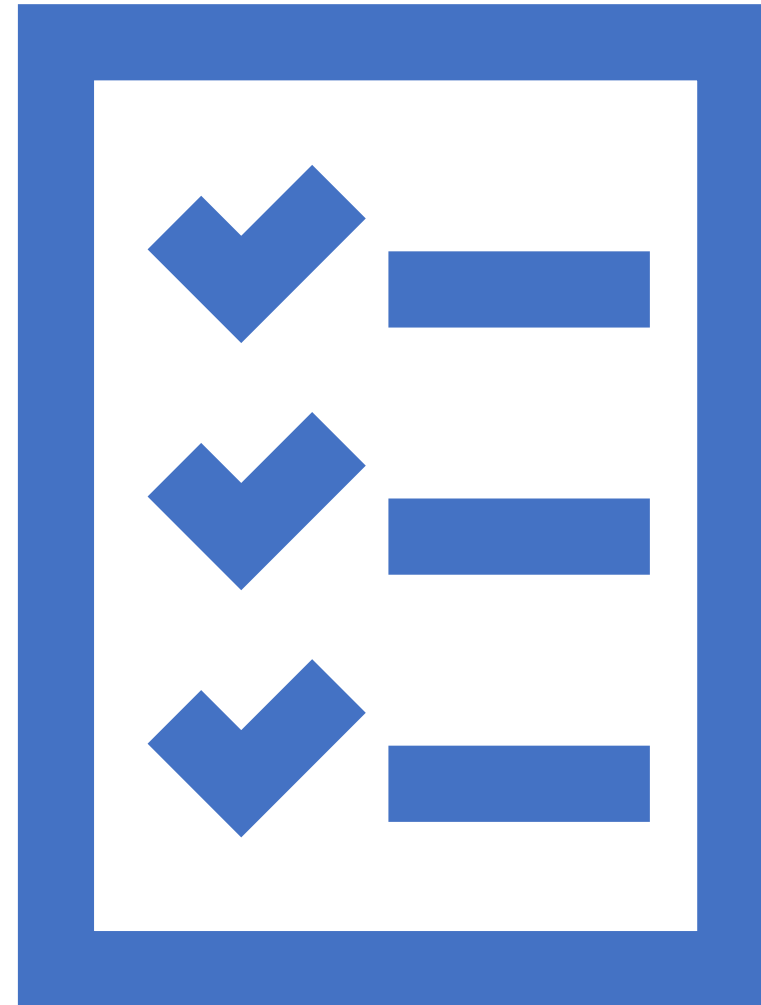
automated tests
pass or fail

how long it takes to
execute pipelines
and builds

Self-serve
telemetry
infrastrucutre



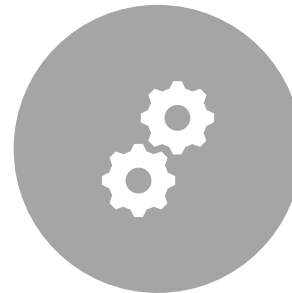
“Monitoring is so important that our monitoring systems need to be more available and scalable than the systems being monitored” – Adrian Cockcroft



Telemetry Users



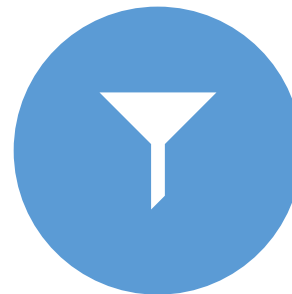
Developers – generate more telemetry to diagnose a problem locally



Ops Engineers – diagnose a production problem



Infosec and auditors - review the telemetry to confirm the effectiveness of a required control



Product manager may use them to track business outcomes, feature usage, or conversion rates

Logging Levels

DEBUG: Information at this level is about anything that happens in the program, most often used during debugging

INFO: Information at this level consists of actions that are user-driven or system specific (e.g., “beginning credit card transaction”)

WARN: Information at this level tells us of conditions that could potentially become an error (e.g., a database call taking longer than some predefined time)

ERROR: Information at this level focuses on error conditions (e.g., API call failures, internal error conditions)

FATAL: Information at this level tells us when we must terminate (e.g., a network daemon can’t bind a network socket)

All potentially significant application events generate logging entries

authentication/autho-
rization decisions

system and data
access

system and
application changes

data changes, such as
adding, editing, or
deleting data

invalid input

resources (RAM, disk,
CPU, bandwidth)

health and availability

startups and
shutdowns

faults and errors

circuit breaker trips

delays

backup
success/failure

Use Telemetry to Guide Problem Solving



Telemetry enables us to use the scientific method to formulate hypotheses about what is causing a particular problem and what is required to solve it



Questions we can answer:

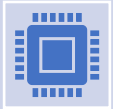
What evidence do we have from our monitoring that a problem is actually occurring?

What are the relevant events and changes in our applications and environments that could have contributed to the problem?

What hypotheses can we formulate to confirm the link between the proposed causes and effects?

How can we prove which of these hypotheses are correct and successfully affect a fix?

Enable Creation of Production Metrics as Part of Daily Work



Create the infrastructure and libraries necessary to make it as easy as possible for anyone in Development or Operations to create telemetry for any functionality they build



It should be as easy as writing one line of code to create a new metric



Open Telemetry



Problems in design and operations can be revealed

Create Self-Service Access to Telemetry and Information Radiators



Anyone who wants information about any of the services we are running can get it without needing production system access or privileged accounts, or having to open up a ticket and wait for days for someone to configure the graph for them



Production metrics will be radiated on web pages generated by a centralized server



Production telemetry to be highly visible



Broadcast information
to internal/external
customers – service
status page => build
and earn customers'
trust



Find and Fill Any Telemetry Gaps



Business level: number of sales transactions, revenue of sales transactions, user sign-ups, churn rate, A/B testing results, etc.



Application level: transaction times, user response times, application faults, etc.



Infrastructure level: web server traffic, CPU load, disk usage, etc.



Client software level: application errors and crashes, user-measured transaction times, etc.



Deployment pipeline level: build pipeline status, change deployment lead times, deployment frequencies, test environment promotions, and environment status



After every production incident, we should identify any missing telemetry that could have enabled faster detection and recovery



The background features two large, curved, overlapping bands. One band is a light blue color, and the other is a light green color. They are positioned in the top right and bottom left corners, framing the central text.

Application and Business Metrics

Application and Business Metrics



Application health



Measure organization goals (# of users, usage of a feature, etc.)

Infrastructure metrics



if a problem occurs in any environment, we can quickly determine whether infrastructure is a contributing cause of the problem



pinpoint exactly what in the infrastructure is contributing to the problem

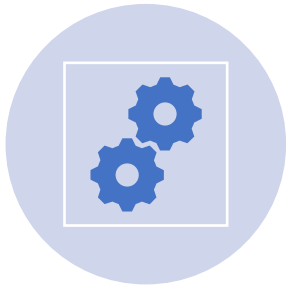


graphing our business metrics alongside our application and infrastructure metrics allows us to detect when things go wrong



measure both Dev and Ops against the real business consequences of downtime: how much revenue should we have attained, but didn't

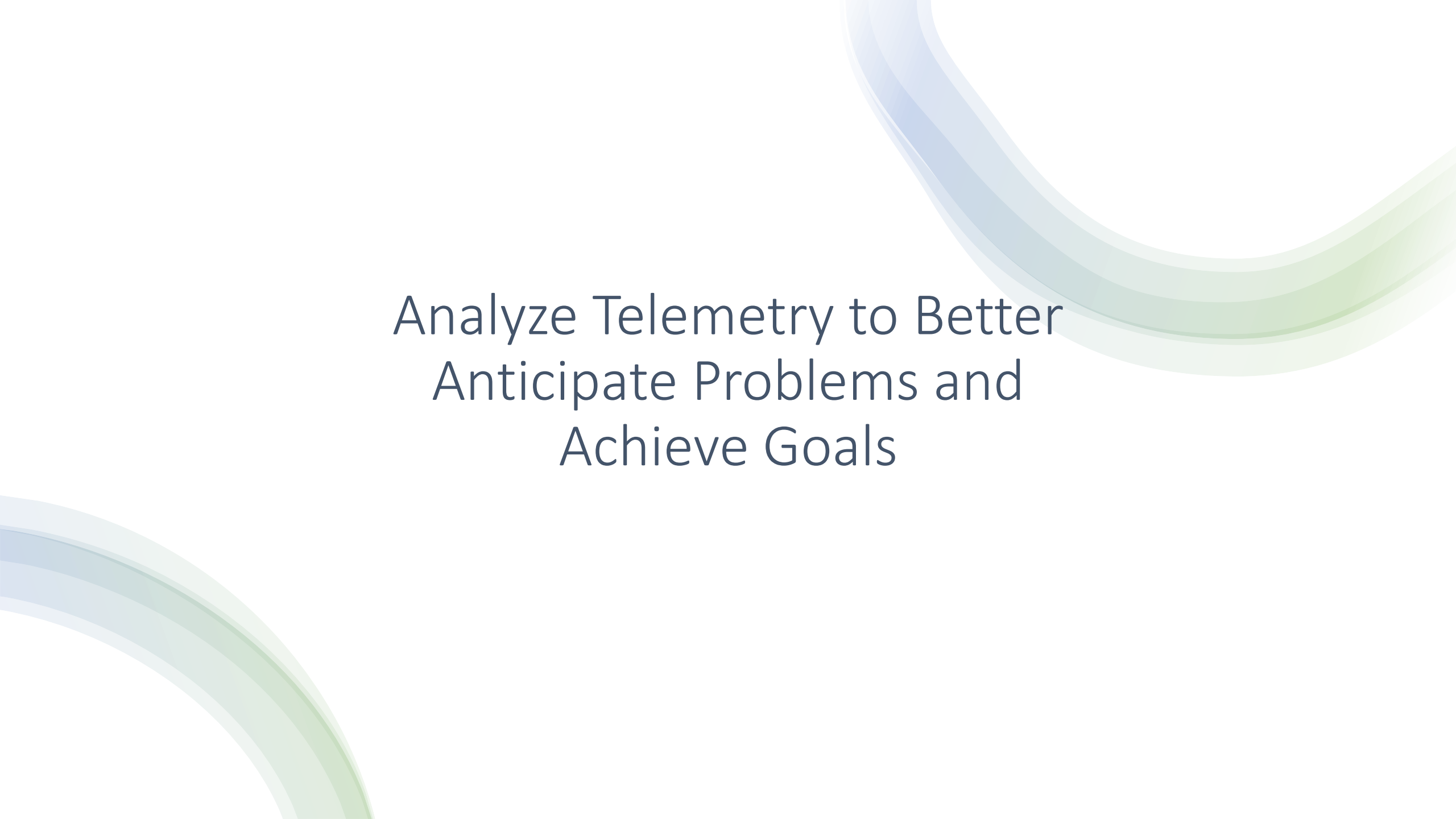
Overlaying Other Relevant Information Onto Our Metrics



All production deployment activities on graphs



Operational activities: when service is under maintenance or being backed up



Analyze Telemetry to Better
Anticipate Problems and
Achieve Goals

Analyze Telemetry to Better Anticipate Problems and Achieve Goals



Proactively find and fix problems
before customers are impacted

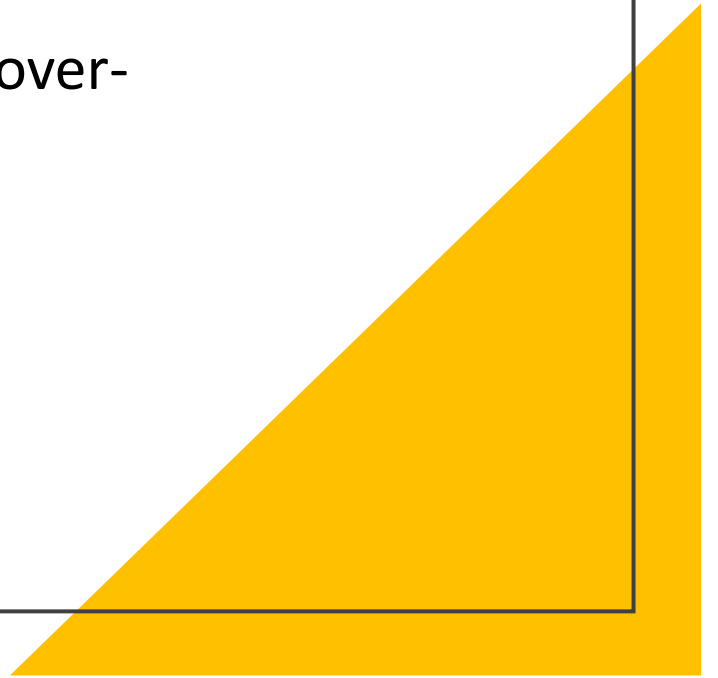


Create more context for
telemetry data to make better
decisions and achieve
organizational goals

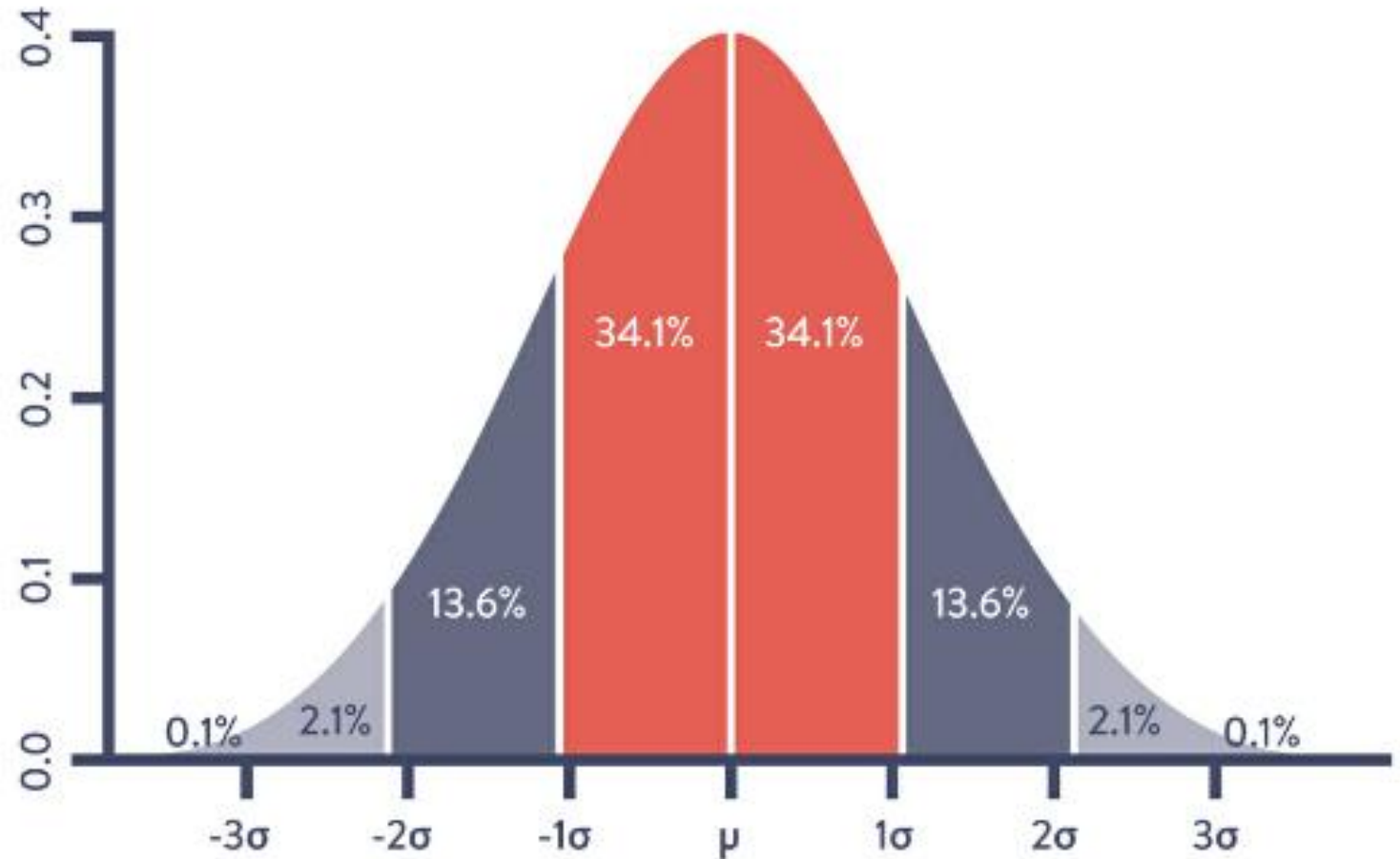
Use Means
and Standard
Deviations to
Detect
Potential
Problems

Avoid alerts that are not actionable
or are false-positives

Avoid under-alerting and over-
alerting



Standard Deviations (σ) & Mean (μ) with Gaussian Distribution



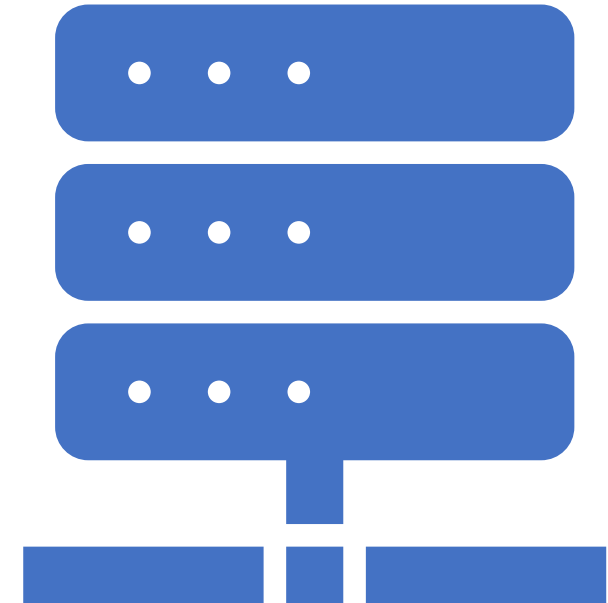
set an alert for when the number of unauthorized login attempts per day is three standard deviations greater than the mean

Instrument and Alert on Undesired Outcomes

- only have alerts that prevent outages, as opposed to being bombarded by alerts after an outage already occurred
- analyze most severe incidents in the recent past (e.g., thirty days) and create a list of telemetry that could have enabled earlier and faster detection and diagnosis of the problem, as well as easier and faster confirmation that an effective fix had been implemented

NGINX web server stopped responding to requests

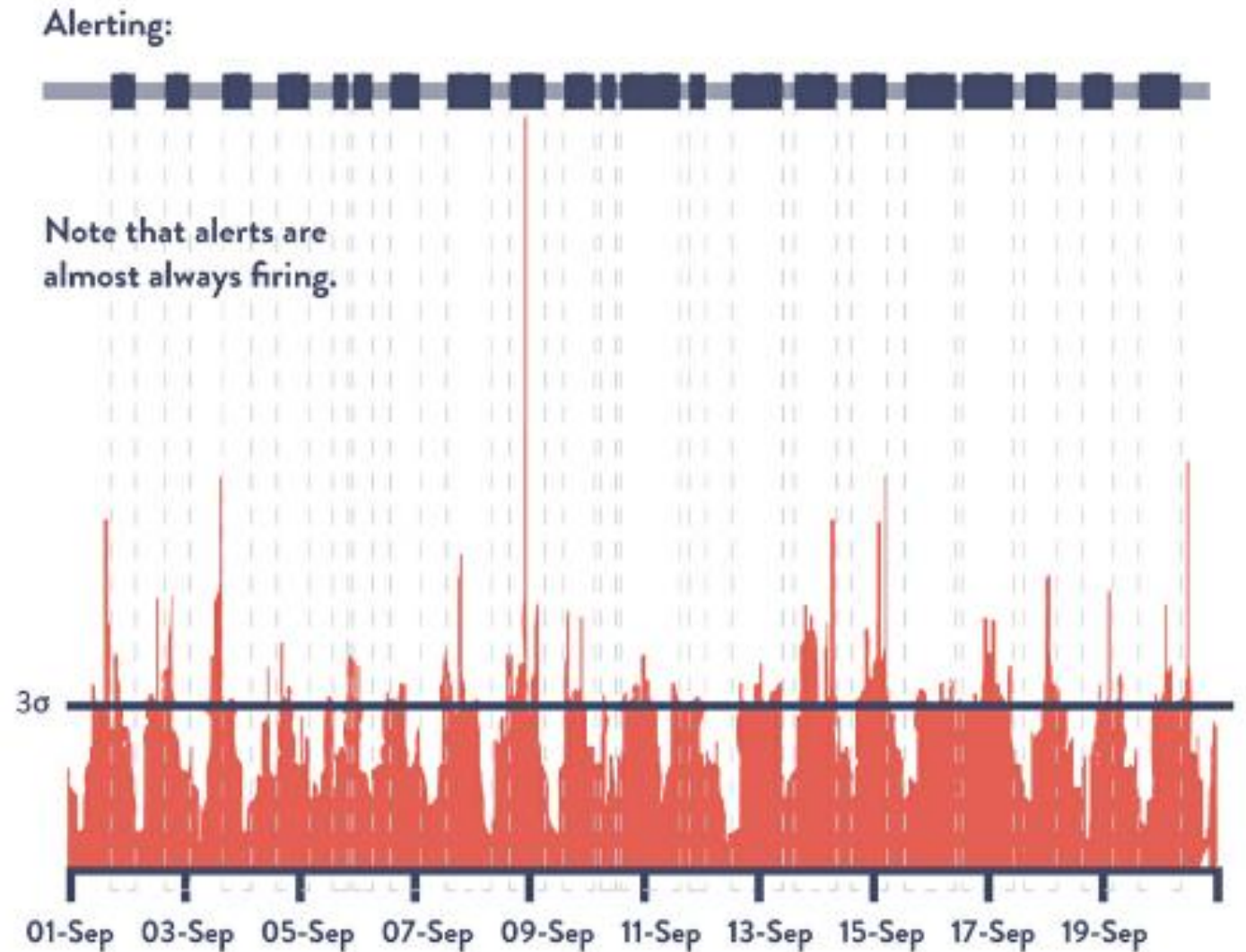
- Application level: increasing web page load times, etc.
- OS level: server free memory running low, disk space running low, etc.
- Database level: database transaction times taking longer than normal, etc.
- Network level: number of functioning servers behind the load balancer dropping



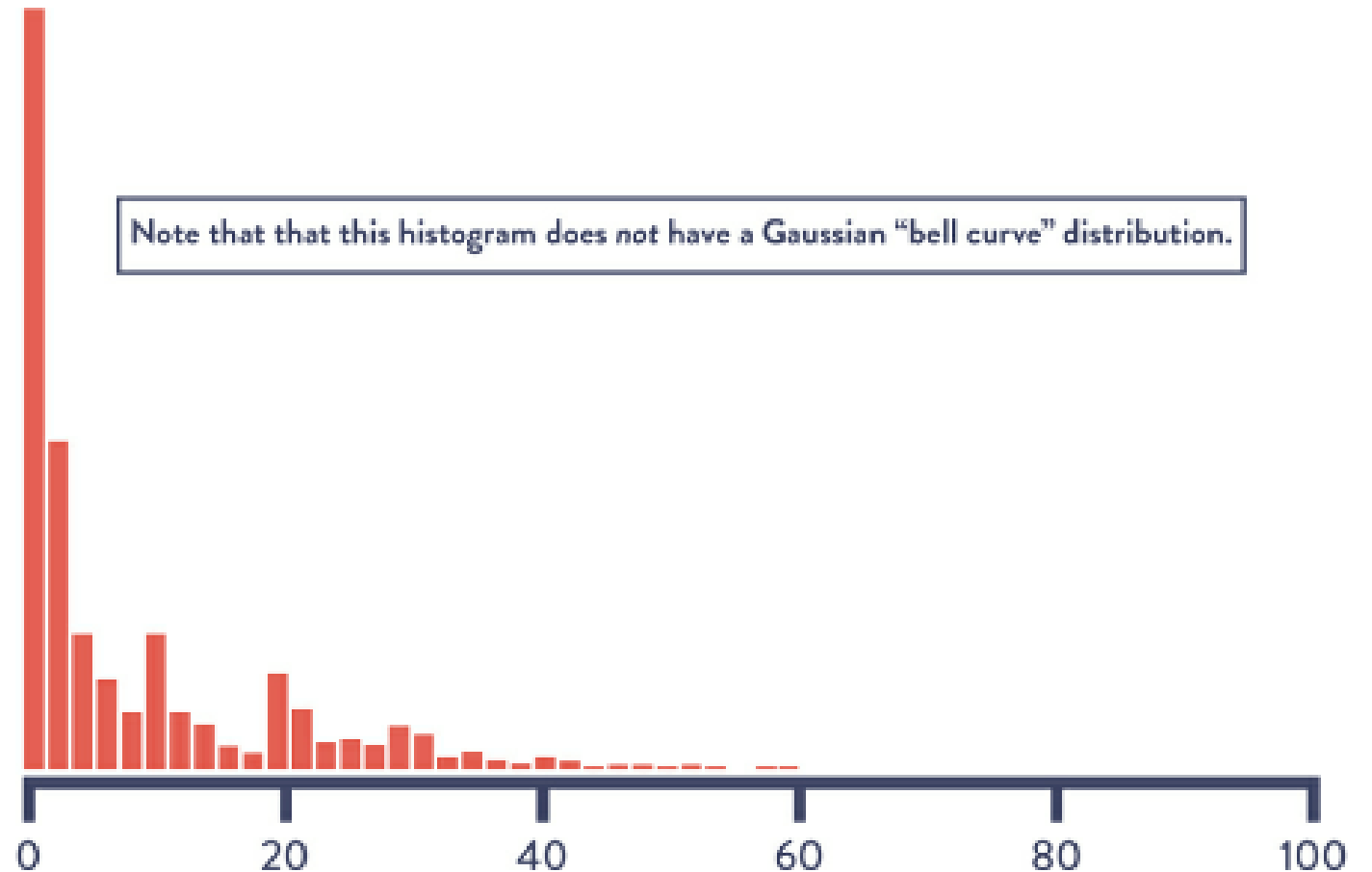
Problems That Arise When Our Telemetry Data Has Non-Gaussian Distribution

Downloads per Minute: Over-Alerting when Using “Three Standard Deviations” Rule

Source: Dr. Toufic Boubez, “Simple math for anomaly detection.”



Downloads per
Minute:
Histogram of
Data Showing
Non-Gaussian
Distribution



Source: Dr. Toufic Boubez, “Simple math for anomaly detection.”



Large percentage of telemetry concerning user data will have periodic/seasonal similarities—web traffic, retail transactions, movie watching, and many other user behaviors have very regular and surprisingly predictable daily, weekly, and yearly patterns.



Using Anomaly Detection Techniques



The search for items or events which do not conform to an expected pattern



Pre-build in tools

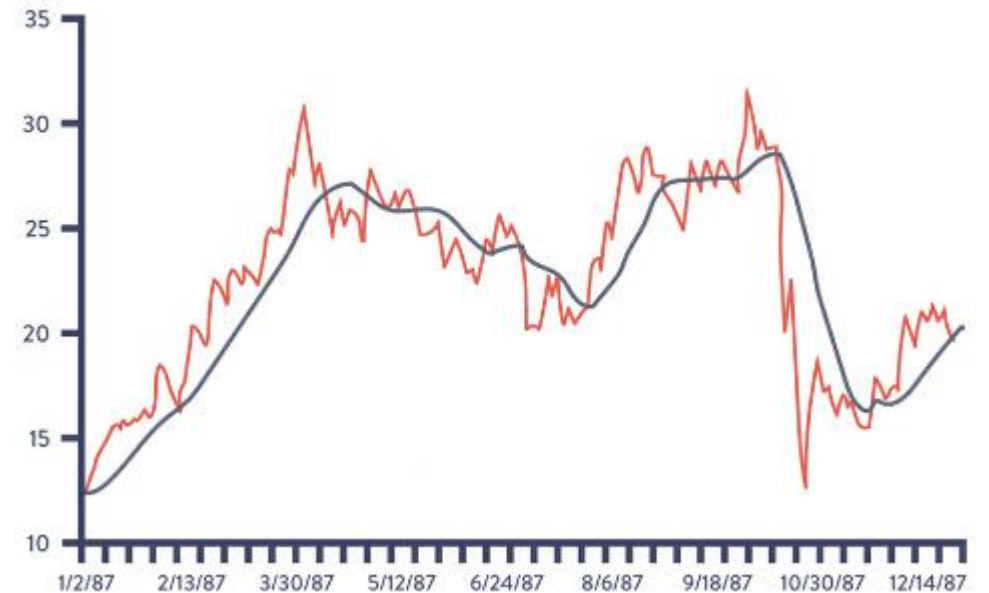


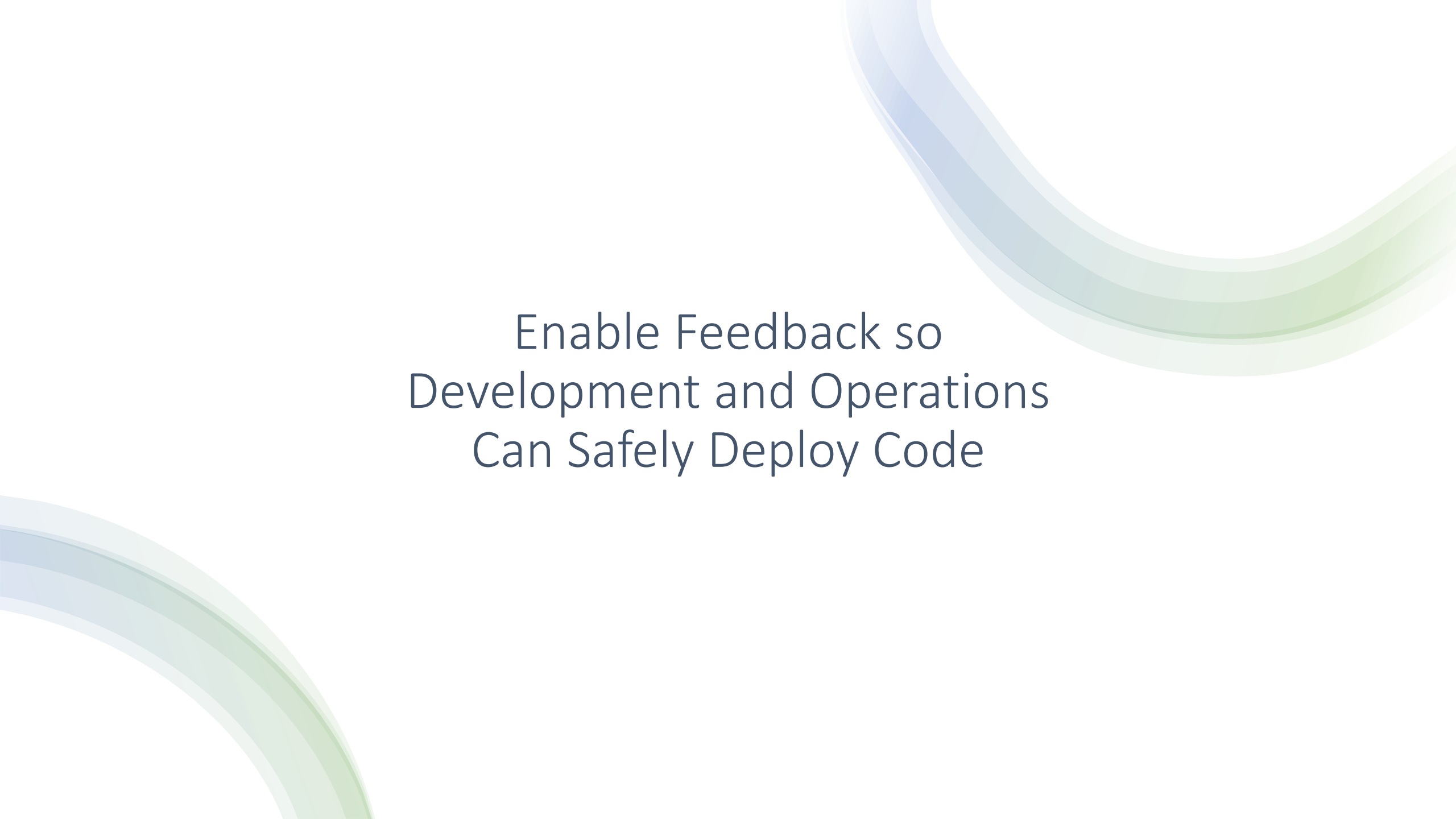
Require statistical analysis of data

Smoothing – moving average

Other techniques

- Fast Fourier Transforms
- Linear Regression
- Kolmogorov-Smirnov Filter (compare 2 series)



The background features two large, curved, overlapping bands of color. One band is a light blue-grey, and the other is a light green. They curve from the top right towards the bottom left, framing the central text.

Enable Feedback so
Development and Operations
Can Safely Deploy Code



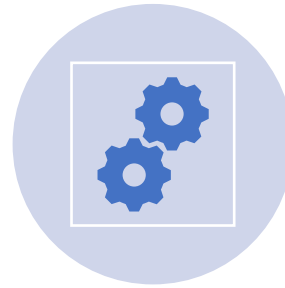
Enable Feedback so Development and Operations Can Safely Deploy Code

- providing faster and more frequent feedback to engineers performing deployments, as well as reducing the batch size of the work, creates safety and confidence
- it is not enough to merely automate the deployment process—we must also integrate the monitoring of production telemetry into our deployment work

Use Telemetry to Make Deployments Safer



ensure that production telemetry is actively monitored when anyone performs a production deployment



quickly determine whether features are operating as designed after the new release is running in production

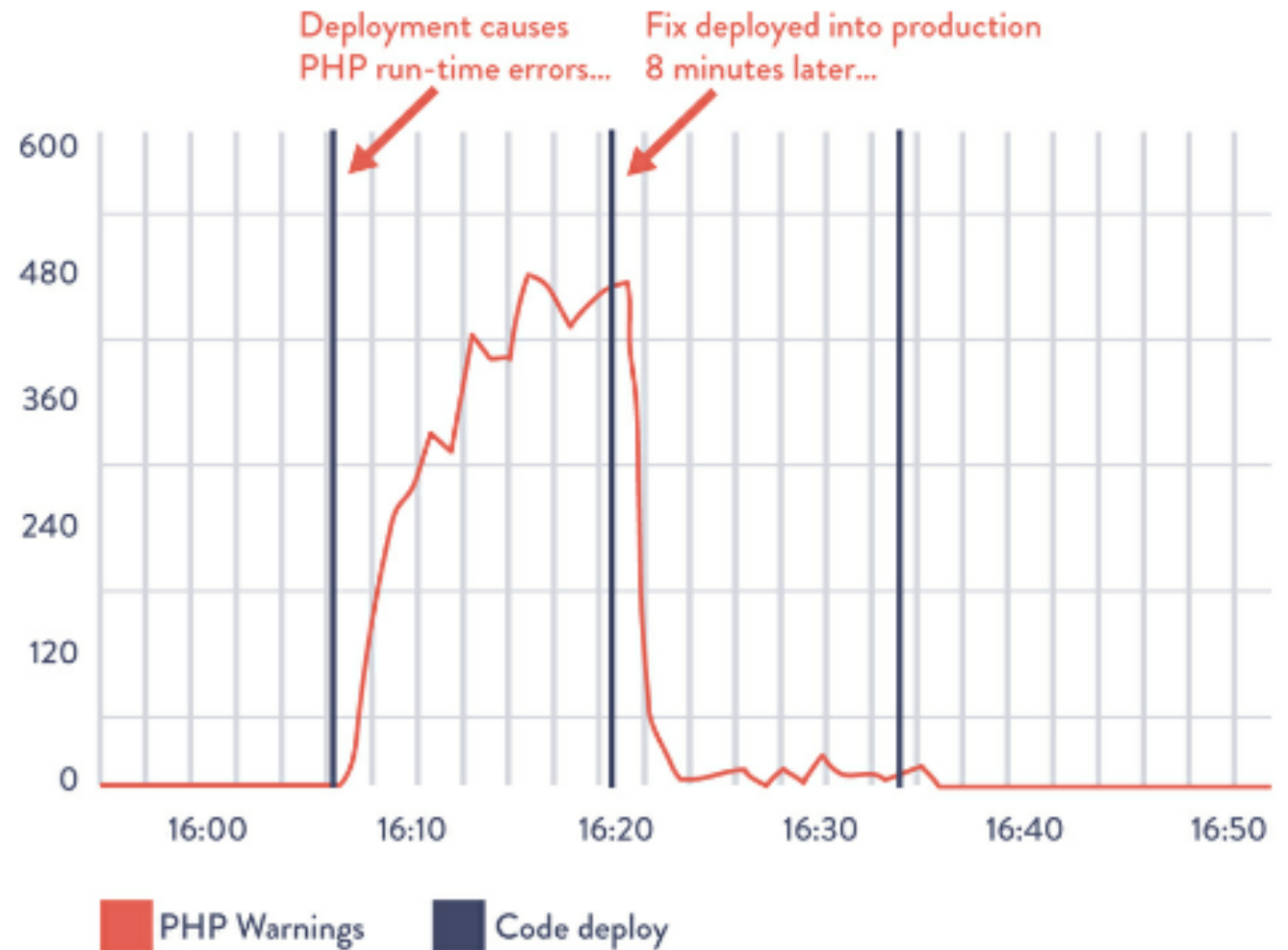
Restore Service After Failed Deploy



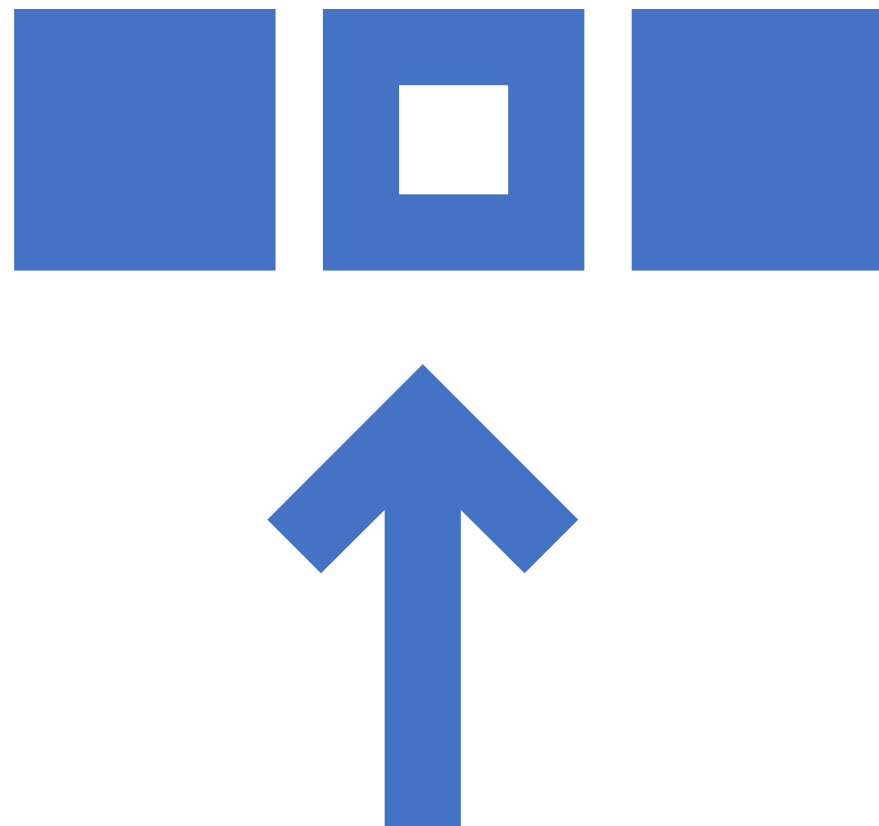
- to turn off broken features with feature toggles
- fix forward
- roll back

Deployment to Etsy.com Causes PHP Run-Time Warnings and Is Quickly Fixed.

Source: Mike Brittain, "Tracking Every Release."



Because production deployments are one of the top causes of production issues, each deployment and change event is overlaid onto the metric graphs to ensure that everyone in the value stream is aware of relevant activity, enabling better communication and coordination, as well as faster detection and recovery



Curs 6





Dev Shares Pager Rotation Duties with Ops

Dev Shares Pager Rotation Duties with Ops



In any complex service there will still be unexpected problems, such as incidents and outages that happen at inopportune times



To ensure everyone in the value stream gets visceral feedback on any upstream architectural and coding decisions they make, developers, development managers, and architects should be on pager rotation



Business goals are not achieved simply because features have been marked as “done.” Instead, the feature is only done when it is performing as designed in production, without causing excessive escalations or unplanned work for either Development or Operations



Have Developers Follow Work Downstream

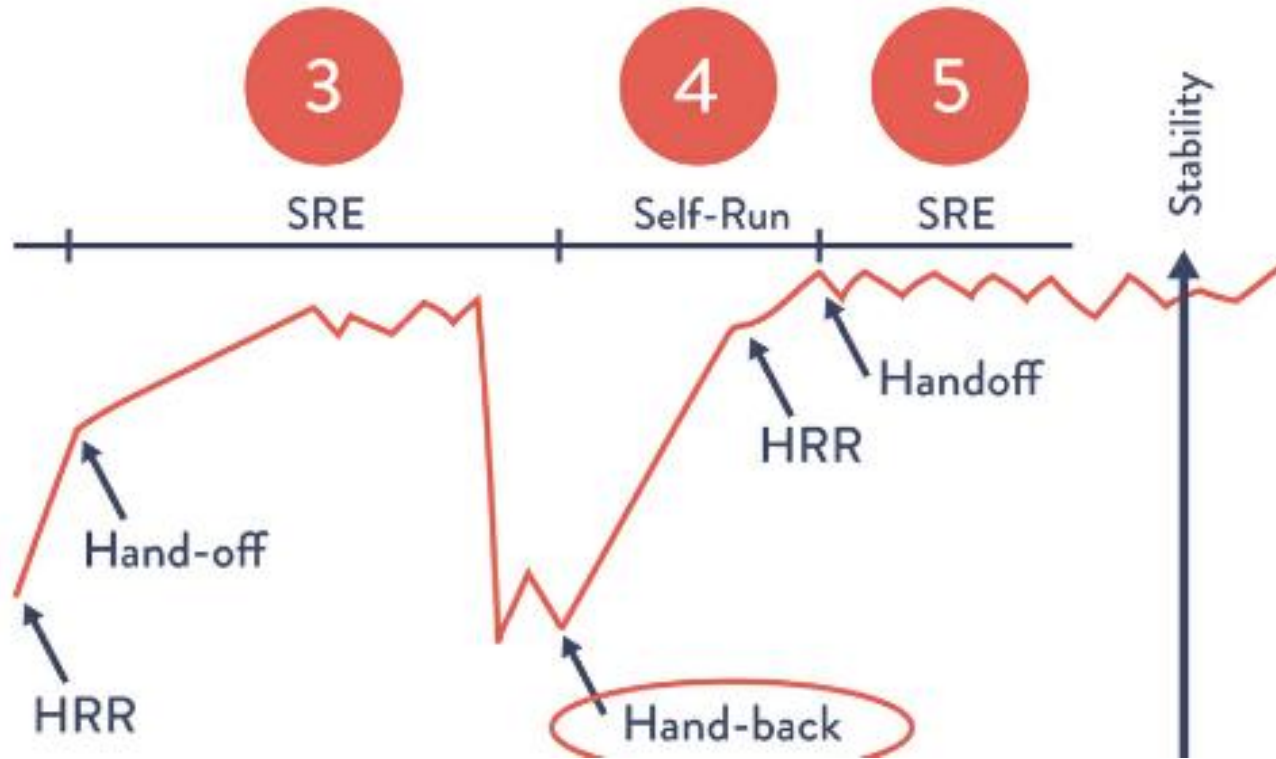
One of the most powerful techniques in interaction and user experience design (UX) is **contextual inquiry**



Have Developers Initially Self-Manage Their Production Service

Launch guidance and requirements will likely include the following:

- **Defect counts and severity:** Does the application actually perform as designed?
- **Type/frequency of pager alerts:** Is the application generating an unsupportable number of alerts in production?
- **Monitoring coverage:** Is the coverage of monitoring sufficient to restore service when things go wrong?
- **System architecture:** Is the service loosely coupled enough to support a high rate of changes and deployments in production?
- **Deployment process:** Is there a predictable, deterministic, and sufficiently automated process to deploy code into production?
- **Production hygiene:** Is there evidence of enough good production habits that would allow production support to be managed by anyone else?



The “Service Handback” at Google

Source: “SRE@Google: Thousands of DevOps Since 2004,” YouTube video, 45:57, posted by USENIX, January 12, 2012, <https://www.youtube.com/watch?v=iluTnhdTzKO>



Integrate Hypothesis-Driven
Development and A/B
Testing into our Daily Work

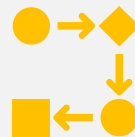
Integrate Hypothesis- Driven Development and A/B Testing into our Daily Work



“The most inefficient way to test a business model or product idea is to build the complete product to see whether the predicted demand actually exists.” – Jez Humble



Before we build a feature, we should rigorously ask ourselves, “Should we build it and why?”



The faster we can experiment, iterate, and integrate feedback into our product or service, the faster we can learn and out-experiment the competition.

A/B Testing

The most commonly used A/B technique in modern UX practice involves a website where visitors are randomly selected to be shown one of two versions of a page, either a control (the “A”) or a treatment (the “B”). Based on statistical analysis of the subsequent behavior of these two cohorts of users, we demonstrate whether there is a significant difference in the outcomes of the two, establishing a causal link between the treatment (e.g., a change in a feature, design element, background color) and the outcome (e.g., conversion rate, average order size)

If we are not performing user research, the odds are that two-thirds of the features we are building deliver zero or negative value to our organization, even as they make our codebase ever more complex, thus increasing our maintenance costs over time and making our software more difficult to change. (Dr. Kohavi)

Integrate A/B Testing into Release



Able to quickly and easily do production deployments on demand



Use feature toggles



Deliver multiple versions of code simultaneously to customer segments



Useful production telemetry at all levels of the application stack

Integrating A/B Testing into Our Feature Planning

Product Owners



think about features as hypotheses and use production releases as experiments with real users to prove or disprove those hypotheses



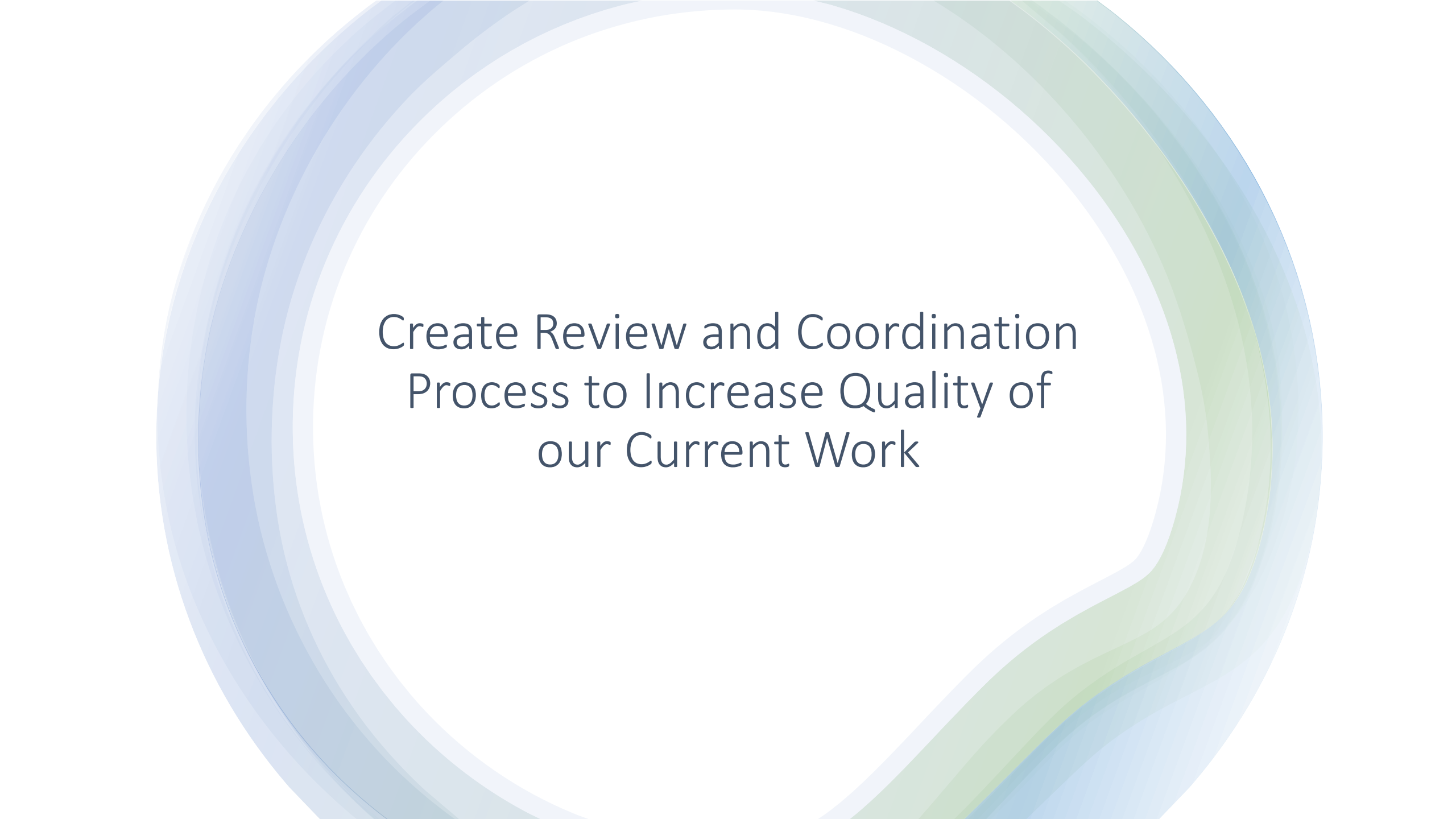
break down work into small units



modify road map of work with alternative paths

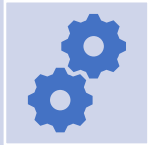
Example

- **We believe** increasing the size of hotel images on the booking page
- **Will result** in improved customer engagement and conversion
- **We will have confidence to proceed** when we see a 5% increase in customers who review hotel images who then proceed to book in forty-eight hours



Create Review and Coordination
Process to Increase Quality of
our Current Work

Create Review and Coordination Process to Increase Quality of our Current Work



Enable Development and Operations to reduce the risk of production changes before they are made



Rely heavily on reviews, inspections, and approvals just prior to deployment



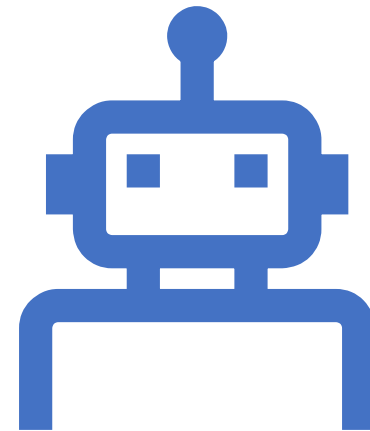
Approvals given by external teams (often too far): hard to make informed decisions and increased change lead times



GitHub Flow

1. To work on something new, the engineer creates a descriptively named branch off of master (e.g., “new-oauth2-scopes”).
2. The engineer commits to that branch locally, regularly pushing their work to the same named branch on the server
3. When they need feedback or help, or when they think the branch is ready for merging, they open a pull request
4. When they get their desired reviews and get any necessary approvals of the feature, the engineer can then merge it into master
5. Once the code changes are merged and pushed to master, the engineer deploys them into production

Ensure that Development,
Operations, and Infosec are
continuously collaborating so that
changes we make to our systems
will operate reliably, securely, safely,
and as designed.



The Dangers of Change Approval Processes



Environments that have low-trust and command-and-control cultures often result in an increased likelihood that problems will occur again, potentially with even worse outcomes



Building high-trust cultures is likely the largest management challenge of this decade

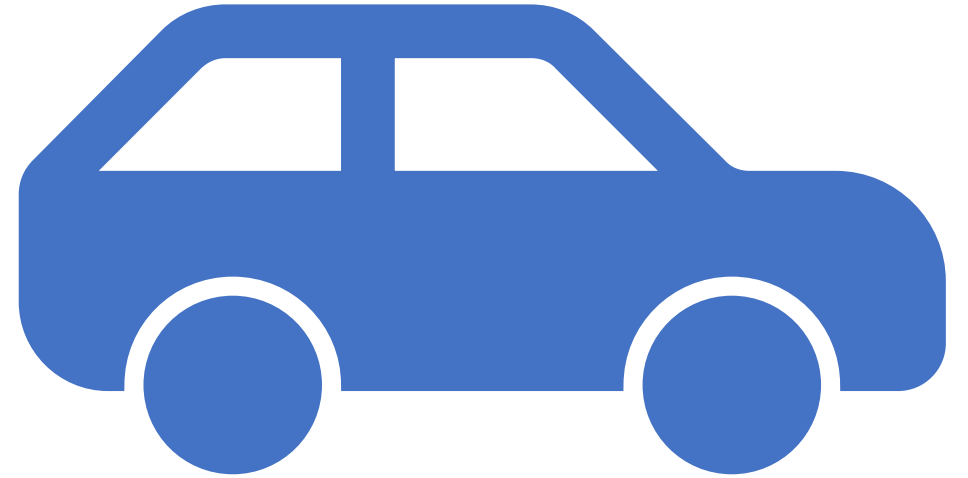
Potential Dangers of “Overly Controlling Changes”

- Traditional change controls can lead to unintended outcomes:
 - long lead times
 - reducing the strength and immediacy of feedback from the deployment process
- These controls often add more friction to the deployment process by multiplying the number of steps and approvals, as well as increasing batch sizes and deployment lead times



Toyota Production System

People closest to a problem typically know the most about it



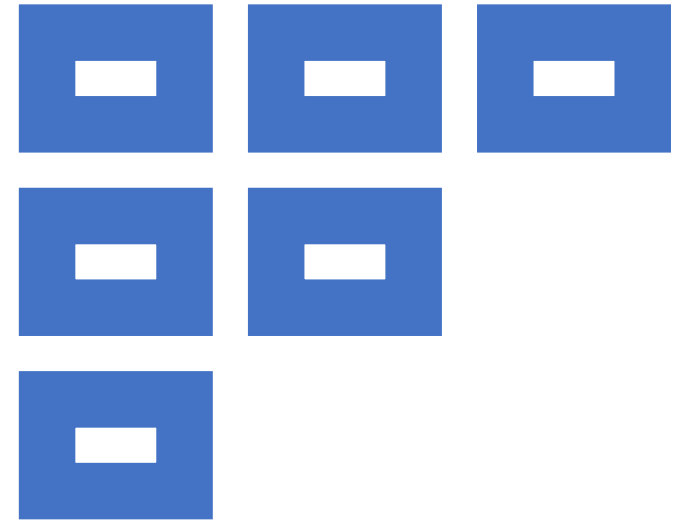
The background of the slide is a close-up, slightly blurred image of a document. It features a line graph with a jagged, upward-trending line. A pen is visible in the upper right corner, resting on the paper. Some numbers are visible on the graph, including '2.5' and '2.47'.

2014 - State of DevOps Report

High-performing teams relied more on peer review and less on external approval of changes

Enable Coordination and Scheduling of Changes

For more complex organizations and organizations with more tightly coupled architectures, we may need to deliberately schedule our changes, where representatives from the teams get together, not to authorize changes, but to schedule and sequence their changes in order to minimize accidents





Enable Peer Review of Changes



Require engineers to get peer reviews of the changes



Applicable to any change (code, infrastructure, etc.)



Improves the quality of the changes



Benefits: cross-training, peer learning, and skill improvement

Peer Review of Changes: Best Practices

Require reviews prior to committing code to trunk in source control

For higher risk areas, such as database changes or business-critical components with poor automated test coverage, we may require further review from a subject matter expert

The principle of small batch sizes also applies to code reviews

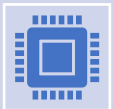
Guidelines for Code Review



everyone must have someone to review their changes (e.g., to the code, environment, etc.) before committing to trunk



Everyone should monitor the commit stream of their fellow team members so that potential conflicts can be identified and reviewed



Define which changes qualify as high risk and may require review from a designated subject matter expert (e.g., database changes, security-sensitive modules such as authentication, etc.)



If someone submits a change that is too large to reason about easily, it should be split up into multiple, smaller changes that can be understood at a glance

Types of code-reviews



Pair programming: Programmers work in pairs



“Over the shoulder:” One developer looks over the author’s shoulder as the latter walks through the code

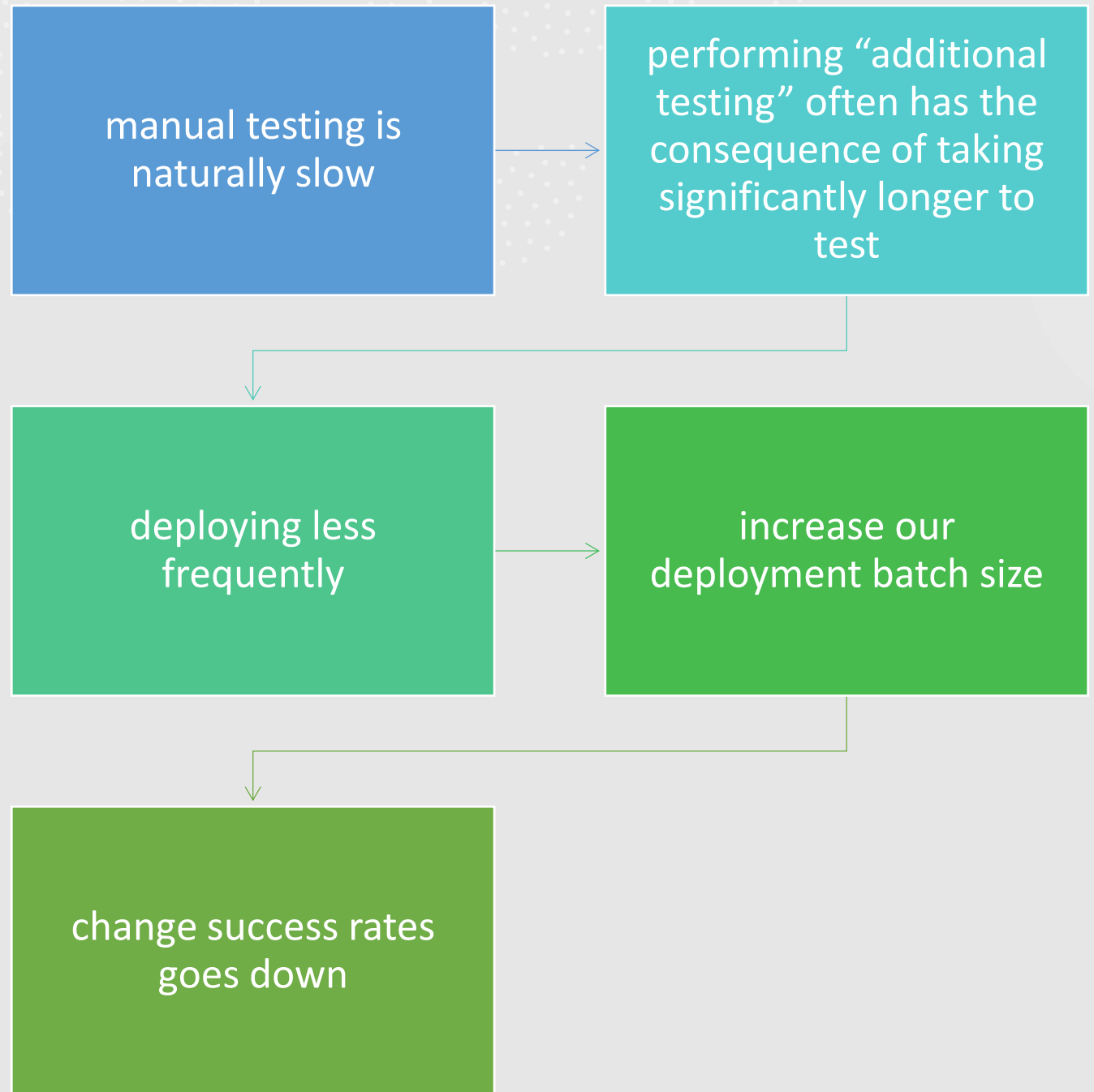


Email pass-around: A source code management system emails code to reviewers automatically after the code is checked in



Tool-assisted code review: Authors and reviewers use specialized tools designed for peer code review (e.g., Gerrit, GitHub pull requests, etc.)

Potential Dangers of Doing More Manual Testing and Change Freezes



How to Avoid More Manual Testing and Change Freezes



fully integrate testing into our daily work as part of the smooth and continual flow into production and increase our deployment frequency



build in quality, which allows us to test, deploy, and release in ever smaller batch sizes.



Pair Programming

Code review on steroids

Spreading knowledge
throughout the organization

Reduce the amount of time
required to get code reviewed

Good Pull Request



there must be sufficient detail on why the change is being made



how the change was made



any identified risks and resulting countermeasures

Good Discussions Around Pull-Requests



additional risks pointed out



ideas on better ways to implement the desired change



ideas on how to better mitigate the risk



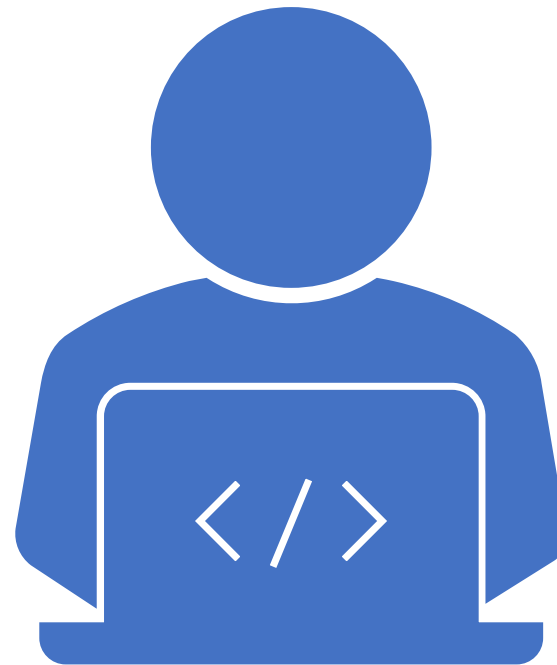
if something bad or unexpected happens upon deployment, it is added to the pull request, with a link to the corresponding issue

Fearlessly Cut Bureaucratic Processes

constantly review processes and
simplify/remove them

How would you answer?

Junior Engineer: Is it okay to deploy a small HTML change?





Engineer is solely responsible for the quality of the change

- Did you have someone review your change?
- Do you know who the best person to ask is for changes of this type?
- Did you do everything you absolutely could to assure yourself that this change operates in production as designed?
- If you did, then don't ask me—just make the change!”